

FUNDAMENTALS OF OOPS

OOPS – Object Oriented Programming System.

What is an Object?

An object is a physical entity with physical existence.

Eg for Objects: Pen, Board, Phone, Laptop, Human etc...

Basics of an Object in Java:

1. **An Object is a block of memory created in the Heap area of JRE memory.**
2. **It can have multiple variables and multiple methods.**
3. **Every object will have an address or reference, which is a random and unique hexadecimal value assigned by the processor.**
4. **Object is a heterogenous collection of data.**
5. **An object is used to represent the real-world object in the software world.**
6. **The properties/attributes of real object are stored in variables and the actions/behaviours of real object are stored in methods.**

Usage of such objects to replicate real world objects in software and develop program codes is called Object Oriented Programming System. Java is a high-level programming language that uses OOPS concepts to build various applications.

Example to understand Object:

If we have to convert a real-world object like a car into a software object, we first have to understand the attributes and behaviour of car.

Attributes/Properties of car:

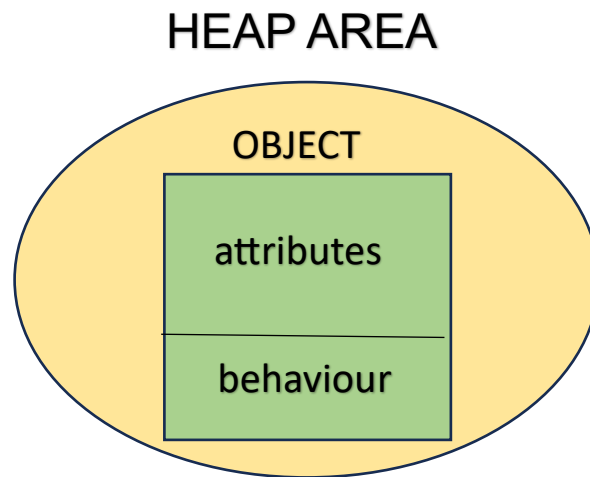
1. Brand
2. Colour
3. Price

Actions/Behaviour of car:

1. Driving
2. Parking

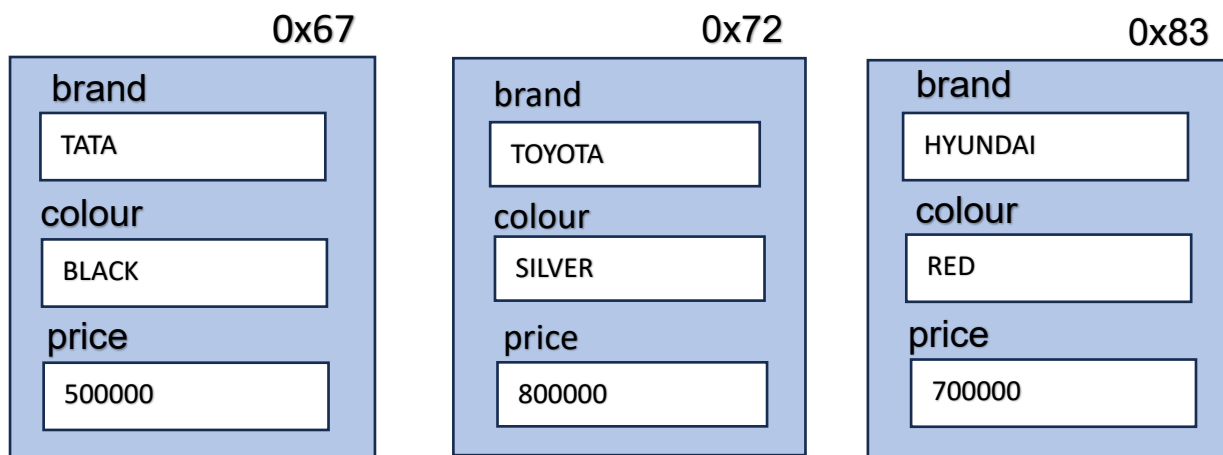
These attributes are stored using non-static variables and behaviours are stored in non-static methods.

Pictorial representation of Object:



How to access the attributes of the car object:

Let us assume we have added three car objects in the program we created.



In the internal heap memory area, it looks as above. To access the data present in these objects, we have to use the address or reference provided to them by the processor. In the above example, 0x67, 0x72 and 0x83 are references assigned.

1. To access the colour of first car object, we have to use,
`System.out.println(0x67.colour);`
2. To access the brand of second car object, we have to use,
`System.out.println(0x72.brand);`
3. To access the price of third car object, we have to use,
`System.out.println(0x83.price);`

So, in all the three cases, the attribute data is accessed by the address or reference. This address is stored in a non-primitive variable called **reference variable** or **object reference**. Using this variable, we can easily access the data members.

CREATION OF OBJECT:

An object is also called as an instance. Hence, creation of object is also called as **Instantiation**. A unary operator called 'new' is used in the process of instantiation. This new keyword creates a memory block in the heap area called as object.

Eg: new ClassName();

In the above example, ClassName() is called as no-argument constructor. This constructor is a special method that has same name as classname. Constructor is used to re-initialize the attributes of the object.

Sequence of steps:

1. new keyword creates a memory block in the heap area.
2. Processor assigns a hexadecimal value to the memory block as address or reference.
3. new keyword returns the address as it is an operator.

As a programmer, it is the coder's responsibility to store the address in a reference variable, which is of non-primitive (ClassName) datatype.

In the above example, let us consider we created a Car class. The object creation or instantiation step will look like:

SYNTAX FOR INSTANTIATION:

Non-primitive_Datatype object_reference = new Classname();

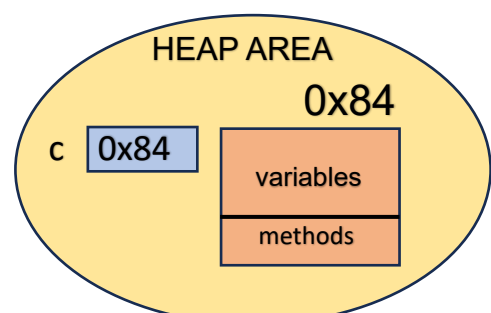
Eg: Car c = new Car();

Here, 'c' is called reference variable that is used to store the address or reference of the object memory block created in the heap memory area. The object reference 'c' is of non-primitive datatype (Car type).

REFERENCE VARIABLE:

The reference variable can store only two types of values:

1. address or reference of another memory block
2. null



CONSTRUCTOR CALLING STATEMENT:

The Constructor has the same name as the ClassName. The Constructor calling statement works similar to method calling statement. Constructor binding happens similar to method binding.

Purpose of Constructor: to load all the non-static members of the class in to the object that is created.

Pre-requisites of Instantiation/Object creation:

1. Blueprint (class) – class with attributes (variables) and behaviours (methods)
 - Use an existing class
 - Create a class
2. Constructor for the class
3. new keyword/operator

It is not mandatory to have methods, but it is recommended to have attributes.

Now let us create a Car class, declare variables and methods, create object for the Car class and access the members using the object reference.

```
1 package training;
2
3 class Car {
4     String brand;
5     String color;
6     double price;
7     public void driving() {
8         System.out.println("Car can be driven");
9     }
10    public void parking() {
11        System.out.println("Car can be parked");
12    }
13    public static void main(String[] args) {
14        Car c = new Car();
15        System.out.println(c.brand);
16        System.out.println(c.color);
17        System.out.println(c.price);
18        System.out.println("Initialize the non-static variables: ");
19        c.brand = "Honda";
20        c.color = "White";
21        c.price = 500000;
22        System.out.println(c.brand);
23        System.out.println(c.color);
24        System.out.println(c.price);
25        c.driving();
26        c.parking();
27    }
28 }
29 }
30
```

<terminated> Car (2) [Java Application] C:\Users\naray\.p2\po
null
null
0.0
Initialize the non-static variables:
Honda
White
500000.0
Car can be driven
Car can be parked